The nested block—the one beginning with `it 'is delicious'`—is an *example* of the sandwich's use. (Other test frameworks might call this a *test case*.) As you write specs, you'll tend to keep each example focused on one particular slice of behavior you're testing.

Tests vs. Specs vs. Examples

What's the difference between tests, specs, and examples? They all refer to the code you write to check your program's behavior. The terms are semi-interchangeable, but each carries a different emphasis:

- A *test* validates that a bit of code is working properly.
- A *spec* describes the desired behavior of a bit of code.
- An *example* shows how a particular API is intended to be used.

We'll use all these terms in this book, depending on which aspect of testing we want to emphasize.

Inside the example, you follow the Arrange/Act/Assert pattern: set up an object, do something with it, and check that it behaved the way you wanted.[2] Here, you create a `Sandwich`, ask it for its `taste`, and verify that the result is `delicious`.

The line beginning with `expect` is an *expectation*. These are like assertions in other test frameworks, but (as we will see later) with a few more tricks up their sleeve.

Take one more look at the three RSpec methods we use in this snippet:

- `RSpec.describe` creates an example group (set of related tests).
- `it` creates an example (individual test).
- `expect` verifies an expected outcome (assertion).

These are the building blocks that you'll reach for again and again as you build your test suites.

Getting the Most Out of RSpec

The specs for your sandwich serve two purposes:

- Documenting what your sandwich should do
- Checking that the sandwich does what it's supposed to

We'd argue that this spec suits the first purpose pretty well. Even someone new to the project can read this code and see that sandwiches should be delicious.

2. http://xp123.com/articles/3a-arrange-act-assert/